

MecaniQA API — OAT 1.

Desenvolvimento Web Orientado a

Objetos

Autores: Arthur Machado Morais
Felipe Souza Teixeira da Silva
Ian da Silva Borges
Jonatan de Souza Ferreira
Victor Tadeu Araújo Augusto

Agenda

O objetivo desta apresentação é demonstrar o desenvolvimento de uma API REST para a MecaniQA, aplicando conceitos de Orientação a Objetos e modelagem de dados para o gerenciamento de peças e serviços.

1. Modelagem dos Dados

Estruturação das classes Peca, Servico e CategoriaPeca.

2. Enum e Padronização

Uso de CategoriaPeca para controlar e validar as categorias das peças.

3. Singleton e Armazenamento

Repositories manuais com instância única e armazenamento temporário em memória.

4. Controllers e API REST

Integração dos controllers com os repositories e implementação das operações CRUD.

5. Validação e Entrega

Status HTTP, testes no Postman, UML, README e validações de QA.

Modelagem dos Dados



A **modelagem dos dados** foi estruturada a partir das necessidades da MecaniQA, utilizando os princípios de Orientação a Objetos para representar as principais informações da API. Foram criadas duas classes principais: **Peça**, responsável pelos dados das peças disponíveis no catálogo e estoque, e **Serviço**, responsável pelos serviços oferecidos pela oficina. Para complementar essa estrutura, foi criada a enumeração **CategoriaPeça**, utilizada para definir categorias padronizadas para cada peça.

A classe **Peça** concentra informações como código, código de barras, fornecedor ou marca, quantidade em estoque, preços de custo e venda, datas de cadastro e atualização, além de características opcionais como tamanho e cor. Já **Serviço** armazena informações como código, nome do serviço, tempo estimado, custo tabelado e datas de criação e atualização.



Peca

CLASSE

Representa as peças disponíveis no catálogo e estoque da MecaniQA.

- # código (gerado pela aplicação)
- |||| códigoBarras
- fornecedor / marca
- quantidadeEstoque
- preçoCusto e preçoVenda
- dataCadastro e dataUltimaAtualizacao
- tamanho e cor (opcionais)
- categoria (obrigatória)

Controle completo do catálogo de peças.



CategoriaPeca

ENUM

Padroniza as categorias disponíveis para classificação das peças, garantindo consistência no cadastro.

MOTOR

SUSPENSAO

FREIOS

ELETRICA

ACESSORIOS

Valores controlados e previsíveis.



Servico

CLASSE

Representa os serviços oferecidos pela oficina MecaniQA.

- # código (gerado pela aplicação)
- nome do serviço
- tempoEstimadoMinutos
- custoTabelado
- dataCriacao e dataUltimaAtualizacao

Gestão organizada dos serviços da oficina.

Enum e Padronização



A enumeração **CategoriaPeca** foi criada para **padronizar a classificação das peças** cadastradas na API. Em vez de permitir que cada categoria seja informada livremente como texto, o sistema trabalha com um conjunto previamente definido de valores: MOTOR, SUSPENSÃO, FREIOS, ELÉTRICA e ACESSÓRIOS.

Essa abordagem melhora a **consistência e a organização dos dados**, evitando diferentes formas de representar uma mesma categoria. O campo **categoria** faz parte da classe Peça e é **obrigatório no cadastro**, garantindo que toda peça esteja associada a uma das categorias previstas pelo sistema.



CategoriaPeca

ENUM

MOTOR

SUSPENSAO

FREIOS

ELETRICA

ACESSORIOS



Por que utilizar Enum?

Garante padronização, validação e consistência na classificação das peças.

Padronização

Valores definidos



Validação

Evita erros de cadastro



Consistência

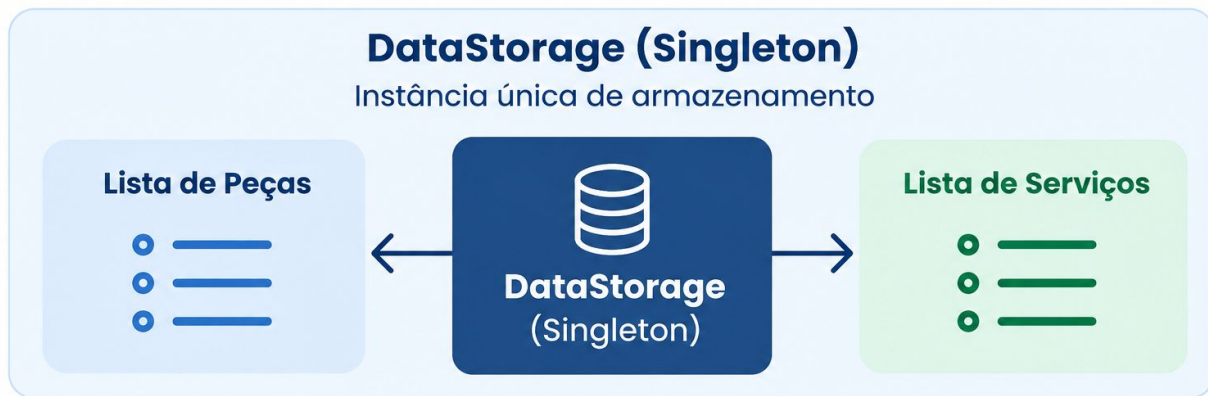
Dados organizados e confiáveis

Singleton e Armazenamento



O padrão **Singleton** foi utilizado nos repositories da aplicação para garantir a existência de uma única instância responsável pelo gerenciamento dos dados de **Peca** e **Servico**. Dessa forma, os controllers acessam sempre o mesmo repository, evitando a criação de diferentes instâncias durante a execução da API.

Os dados são armazenados temporariamente **em memória**, utilizando estruturas do tipo **List<T>**. Os repositories são responsáveis pelas operações de cadastro, consulta, atualização e remoção, funcionando como a camada intermediária entre os controllers e os dados da aplicação.



Principais Benefícios



Instância única

Evita múltiplas cópias de dados.



Acesso centralizado

Todas as operações usam o mesmo dados.



Armazenamento em memória

Dados disponíveis durante a execução da aplicação.

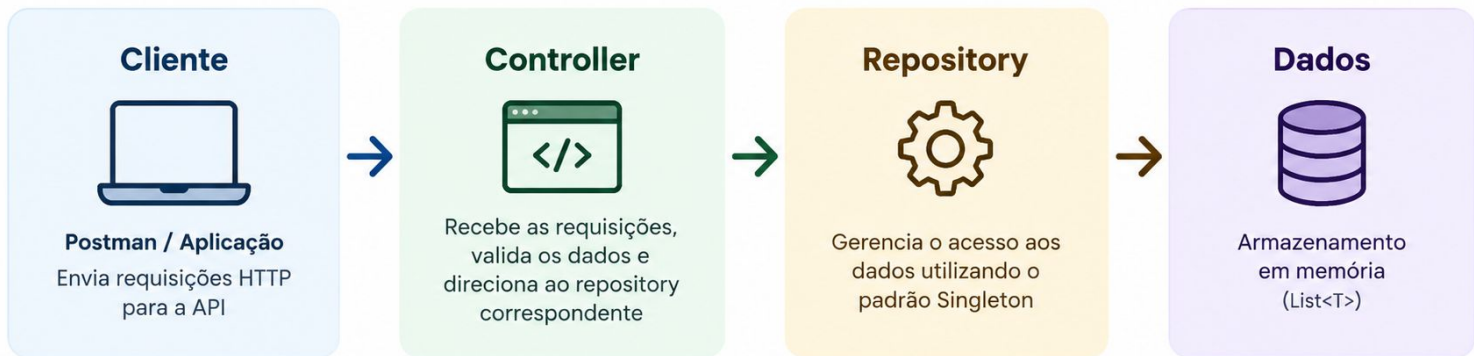
Controllers e API Rest



Os **Controllers** são responsáveis por receber as requisições HTTP realizadas à API e direcionar cada operação ao repository correspondente. No projeto, essa integração foi realizada manualmente utilizando o método **getInstance()** do Singleton, sem a utilização de **Autowired**.

A API implementa as principais operações de **CRUD** para o gerenciamento de peças e serviços. Foram utilizados os métodos **POST** para cadastrar, **GET** para consultar, **PUT** para atualizar e **DELETE** para remover registros.

Cada operação retorna uma resposta HTTP adequada ao resultado da requisição, permitindo identificar situações de sucesso, criação de registros, dados inválidos ou recursos não encontrados.



Principais Endpoints (CRUD)

POST

Cadastrar

Cria um novo registro de peça ou serviço.

Ex.: POST /pecas
 Ex.: POST /servicos

GET

Consultar

Retorna os registros de peças ou serviços.

Ex.: GET /pecas
 Ex.: GET /servicos

PUT

Atualizar

Atualiza um registro existente.

Ex.: PUT /pecas/{id}
 Ex.: PUT /servicos/{id}

DELETE

Remover

Remove um registro de peça ou serviço.

Ex.: DELETE /pecas/{id}
 Ex.: DELETE /servicos/{id}

Validação e Entrega



A etapa de **validação** teve como objetivo verificar o funcionamento das operações implementadas na API. Os endpoints de **Peça** e **Serviço** foram testados por meio do **Postman**, permitindo validar as operações de cadastro, consulta, atualização e remoção.

Também foram verificados os **status HTTP** retornados pela aplicação, contemplando tanto os cenários de sucesso quanto situações de dados inválidos ou registros não encontrados.

Como parte da entrega, o projeto foi complementado com o **diagrama UML** para representar a estrutura das classes e com o **README**, contendo as principais informações sobre o projeto, sua execução e os endpoints disponíveis.

Considerações finais



O desenvolvimento da **MecaniQA API** permitiu aplicar na prática conceitos fundamentais de **Orientação a Objetos** e desenvolvimento de APIs REST. A solução foi estruturada a partir da modelagem de **Peça** e **Serviço**, utilização de **Enum**, aplicação do padrão **Singleton** e implementação das operações de **CRUD**.

A separação entre **Controllers**, **Repositories** e modelos contribuiu para uma estrutura mais organizada e de fácil compreensão. Os testes realizados também permitiram validar o comportamento dos endpoints e os retornos HTTP da aplicação.

Referências



https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods?utm_source=chatgpt.com

https://learning.postman.com/docs/?utm_source=chatgpt.com

<https://www.thoughtco.com/what-is-an-enum-958326>

<https://refactoring.guru/pt-br/design-patterns/singleton>